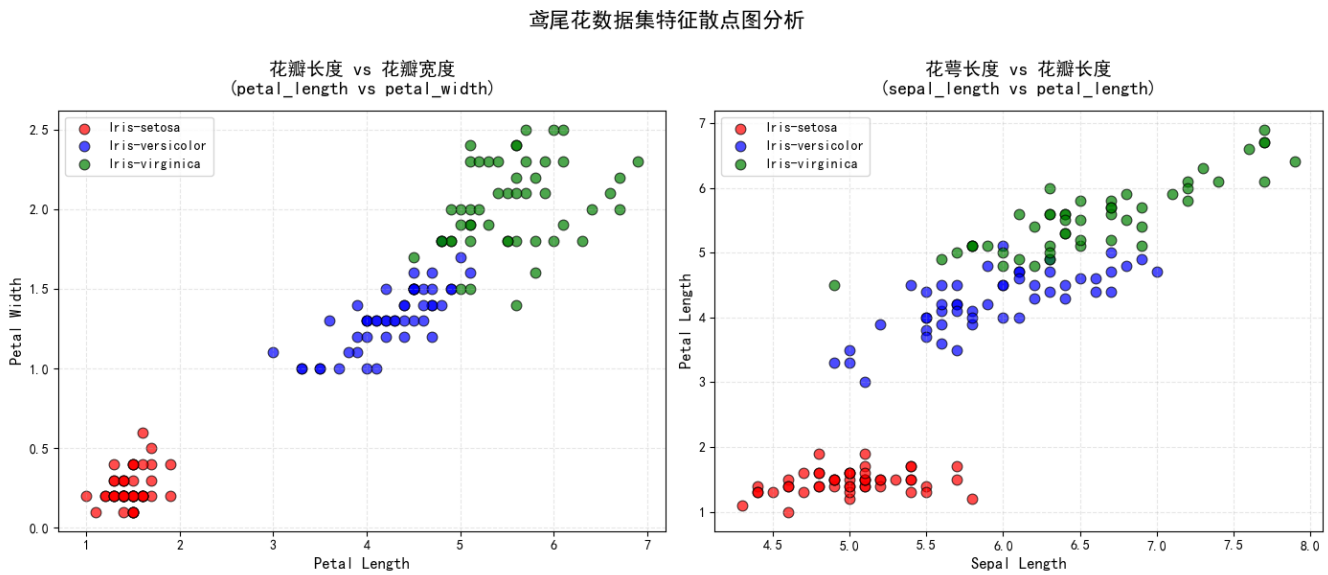
鸢尾花分类

1. 目的

对鸢尾花数据进行分析并训练模型，在测试集上预测并验证模型的准确性。

2.模型选取与数据预处理

已知三类标签中有一类与其他两类线性可分，另外两类不可分，且花瓣长度、花瓣宽度与类别的相关系数较高，于是选取（petal_length，petal_width）和（sepal_length，petal_length）两组特征绘图确定线性可分的类别：



显然，红色的 Iris-setosa 特征是线性可分的，由于高斯核支持向量机在带宽较大时接近线性，同时也可以学习非线性关系，所以我们选取高斯核的支持向量机作为我们分类的模型，作为比较我们额外选取线性的 Softmax 回归进行这个多分类问题。

由于 sklearn 中的 SVM 算法会对每一个类别单独训练分类器，标签的数值大小不会产生序关系，所以对标签直接赋以数值。以 8：2 比例分割训练集和测试集后，由于高斯核支持向量机基于样本间的欧式距离，对尺度敏感，对特征进行了标准化操作。

3 算法实现与结果

3.1 训练模型

利用 sklearn 中的 SVM 函数训练模型，超参数选择：`kernel='rbf', C=1.0, gamma='scale', random_state=23`。kernel 选择为'rbf'即选用高斯核；C 为模型复杂度的惩罚项系数；gamma 是核函数$K(x_i, x_j) = \exp(-\gamma \parallel x_i - x_j \parallel^2)$中的系数，选择为默认值'scale':$\gamma = \frac{1}{n_{\text{features}} \cdot \text{Var}(X)}$；随机数种子选取为 23。

利用 sklearn 中的线性模型训练 Softmax 回归模型，超参数选择：`LogisticRegression(multi_class='multinomial', solver='lbfgs', max_iter=1000, random_state=77)`。multi_class 选择为'multinomial'表示使用 Softmax 回归；solver 表示优化算法的类型；max_iter=1000 表示最大迭代次数为 1000；随机种子取 77。

3.2 训练结果

得到训练的结果为：

模型	训练集准确率	测试集准确率
Softmax 回归	0.9583	0.9333
非线性 SVM	0.9750	0.9667

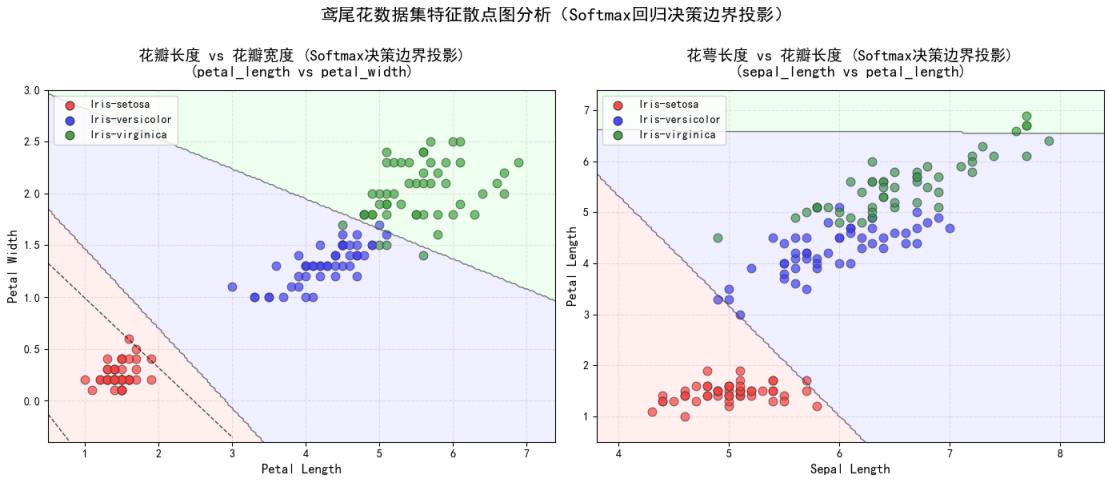
其中，SVM 的混淆矩阵为：

真实 \ 预测	数据集	Iris-setosa	Iris-versicolor	Iris-virginica
Iris-setosa	训练集	40	0	0
	测试集	10	0	0
Iris-versicolor	训练集	0	39	1
	测试集	0	9	1
Iris-virginica	训练集	0	2	38
	测试集	0	0	10

Softmax：

真实 \ 预测	数据集	Iris-setosa	Iris-versicolor	Iris-virginica
Iris-setosa	训练集	40	0	0
	测试集	10	0	0
Iris-versicolor	训练集	0	38	2
	测试集	0	9	1
Iris-virginica	训练集	0	3	37
	测试集	0	1	9

结果表明，对于线性可分的 Iris-setosa 类而言两种模型都能正确的区分出来；而在线性不可分的 Iris-versicolor 和 Iris-virginica 两个类别上，SVM 的表现比 Softmax 略好，泛化能力也更强，验证了非线性模型在线性不可分的分类任务中比线性模型表现更好。



在 Softmax 中我们可以把分类边界在两个平面上的投影画出来，由图可以看出（petal_length，petal_width）属性对确实是较好的分类依据，也可以看出 Softmax 在边界点才出现混淆问题，能够准确判别非边界的点。

4.AI 使用说明

绘图部分的代码使用了 Deepseek：

算了就两组吧，分别画子图在一个图里就行，只要一个方案

2 / 2

已思考（用时 3 秒）

好的，我推荐用**花瓣长度 vs 花瓣宽度 (petal_length vs petal_width)** 这组特征，通常最能显示Iris-setosa与其他两类的线性可分性。这是包含中文适配的绘图代码：

python 复制 下载

```python
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# 设置中文字体，解决中文显示问题
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False

# 定义两组特征组合
feature_pairs = [
    ('petal_length', 'petal_width', '花瓣长度 vs 花瓣宽度'),
    ('sepal_length', 'petal_length', '花萼长度 vs 花瓣长度')
]
```

import pandas as pd import numpy

print()

请给上述代码做可视化，要求将softmax的分类边界画在二维图上

我注意到你的代码使用了所有4个特征进行训练，但要在二维图上展示分类边界，我们需要选择或合成两个主要特征。以下是修改后的代码，包含多种可视化方案：

python 复制 下载

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.decomposition import PCA
import matplotlib.patches as mpatches

# 1. 读取数据
filename = r"C:\Users\29414\Desktop\数据挖掘作业\鸢尾花\IRIS.csv"
df = pd.read_csv(filename)

# 2. 标签编码
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(df['species'])
```

5.代码

绘制图像部分:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu
Sans']
plt.rcParams['axes.unicode_minus'] = False
#读取数据
filename=r"C:\Users\29414\Desktop\数据挖掘作业\鸢尾花\IRIS.csv"
df = pd.read_csv(filename)


# 绘图寻找特征（此段代码来自 Deepseek）
feature_pairs = [
    ('petal_length', 'petal_width', '花瓣长度 vs 花瓣宽度'),
    ('sepal_length', 'petal_length', '花萼长度 vs 花瓣长度')
]

# 定义颜色和标记
colors = ['red', 'blue', 'green']
species_list = df['species'].unique()

# 创建子图
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# 遍历两组特征组合
for idx, (x_feat, y_feat, title) in enumerate(feature_pairs):
    ax = axes[idx]

    # 绘制每个类别的散点
    for i, species in enumerate(species_list):
        species_data = df[df['species'] == species]
        ax.scatter(
            species_data[x_feat],
            species_data[y_feat],
            c=colors[i],
            s=60,
            alpha=0.7,
            edgecolors='black',
            linewidth=0.8,
            label=species
        )

    # 设置标题和标签
    ax.set_title(f'{title}\n({x_feat} vs {y_feat})', fontsize=14,
fontweight='bold', pad=12)
    ax.set_xlabel(x_feat.replace('_', ' ').title(), fontsize=12)
    ax.set_ylabel(y_feat.replace('_', ' ').title(), fontsize=12)

    # 添加网格
    ax.grid(True, alpha=0.3, linestyle='--')
```

```
    # 添加图例
    ax.legend(loc='upper left', fontsize=10)

# 调整整体标题
fig.suptitle('鸢尾花数据集特征散点图分析', fontsize=16, fontweight='bold',
y=1)

# 调整布局并显示
plt.tight_layout()
plt.show()
```

高斯核 SVM:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import numpy as np

#读取数据
filename = r"C:\Users\29414\Desktop\数据挖掘作业\鸢尾花\IRIS.csv"
df = pd.read_csv(filename)

#数据预处理
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(df['species'])

X = df.drop('species', axis=1)

#分割
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded  # 保持类别比例
)
#归一化
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # 只在训练集上拟合
X_test_scaled = scaler.transform(X_test)       # 用训练集参数转换测试集
#训练 SVM 模型
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, classification_report

svm_model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
svm_model.fit(X_train_scaled, y_train)


y_train_pred = svm_model.predict(X_train_scaled)
y_test_pred = svm_model.predict(X_test_scaled)
```

```
#混淆矩阵
train_cm = confusion_matrix(y_train, y_train_pred)
test_cm = confusion_matrix(y_test, y_test_pred)

class_names = label_encoder.classes_

print("训练集混淆矩阵 (类别顺序: {})".format(", ".join(class_names)))
print(train_cm)
print()

print("测试集混淆矩阵 (类别顺序: {})".format(", ".join(class_names)))
print(test_cm)
print()

print("训练集分类报告:")
print(classification_report(y_train, y_train_pred,
target_names=class_names))
print()

print("测试集分类报告:")
print(classification_report(y_test, y_test_pred,
target_names=class_names))

#计算准确率
train_accuracy = svm_model.score(X_train_scaled, y_train)
test_accuracy = svm_model.score(X_test_scaled, y_test)

print("=" * 60)
print(f"训练集准确率: {train_accuracy:.4f}")
print(f"测试集准确率: {test_accuracy:.4f}")
```

Softmax 分类:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report

#读取数据
filename = r"C:\Users\29414\Desktop\数据挖掘作业\鸢尾花\IRIS.csv"
df = pd.read_csv(filename)

#数据预处理
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(df['species'])

X = df.drop('species', axis=1)
feature_names = X.columns.tolist()
##分割
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
##归一化
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 训练 softmax 模型
softmax_model = LogisticRegression(
    multi_class='multinomial',
    solver='lbfgs',
    max_iter=1000,
    random_state=77
)
softmax_model.fit(X_train_scaled, y_train)

#预测、评估
y_train_pred = softmax_model.predict(X_train_scaled)
y_test_pred = softmax_model.predict(X_test_scaled)

class_names = label_encoder.classes_

#结果
print("=" * 60)
print("Softmax 回归模型评估")
print("=" * 60)
print()

print("训练集混淆矩阵 (类别顺序: {})".format(", ".join(class_names)))
print(confusion_matrix(y_train, y_train_pred))
print()

print("测试集混淆矩阵 (类别顺序: {})".format(", ".join(class_names)))
print(confusion_matrix(y_test, y_test_pred))
print()

print("训练集分类报告:")
print(classification_report(y_train, y_train_pred,
target_names=class_names))
print()

print("测试集分类报告:")
print(classification_report(y_test, y_test_pred,
target_names=class_names))

#准确率
train_accuracy = softmax_model.score(X_train_scaled, y_train)
test_accuracy = softmax_model.score(X_test_scaled, y_test)

print("=" * 60)
```

```python
print(f"训练集准确率: {train_accuracy:.4f}")
print(f"测试集准确率: {test_accuracy:.4f}")
print()
```